Alignerz Mitigation Review

Final Report

Dualguard

January 4, 2026

Alignerz Mitigation Review

Dualguard

December 2025 - January 2026

Prepared by *Dualguard*:

- Lead Security Guards: *0xAlix2*, *0xSorryNotSorry*

Table of Contents

Protocol Summary

AlignerZ is transforming the investment landscape with a secure, transparent, and equitable launchpad designed for long-term investors. They are committed to protecting communities from being treated as exit liquidity. To achieve this, they have developed a sophisticated adaptive system that aligns the interests of projects and their supporters, fostering sustainable growth. Their model not only prevents exploitation, it actively incentivize conviction. The most committed investors are prioritized and rewarded proportionally based on their long-term dedication.

Disclaimer

The *Dualguard* team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security review by the *Dualguard* team is not an endorsement of the underlying business or product. The security review was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H	M
Likelihood	Medium	H	M	L
	Low	M	L	L

Impact

- High: Results in a substantial loss of assets within the protocol or significantly impacts a group of users.
- Medium: Causes a minor loss of funds (such as value leakage) or affects a core functionality of the protocol.
- Low: Leads to any unexpected behavior in some of the protocol's functionalities, but is not critical.

Likelihood

- High: The attack path is feasible with reasonable assumptions that replicate on-chain conditions, and the cost of the attack is relatively low compared to the potential funds that can be stolen or lost.
- Medium: The attack vector is conditionally incentivized but still relatively likely.
- Low: The attack requires too many or highly unlikely assumptions, or it demands a significant stake by the attacker with little or no incentive.

Action required for severity levels

- High: Must fix
- Medium: Should fix
- Low: Could fix

Non-Security Issues

- Info/ QA / Non-Critial: A non-security issue, like a suggested code improvement, a comment, a renamed variable, etc. Security researchers did not attempt to find an exhaustive list of these.
- Gas: Gas saving / performance suggestions. Security researchers did not attempt to find an exhaustive list of these.

N.B:

- Admin mistakes(even when admin is not considered dumb) are at best low severity issues
- If there's a work around to mitigate the vulnerability through function calls, the severity is downgraded once:
  - H => M
  - M => L
  - L => I

Security Review Details

The findings described in this document correspond the following commit hash:

```
e4354a2407ba5358a1ab22298b5712aa224fbb20
```

The fixed version of the codebase is at this commit hash:

```
4e62ea5f120d1630bfdadb6cb921987ef7f5dca5
```

Scope

Between the 23rd of December 2025 and the 2nd of January 2026, the *Dualguard* senior guards that were assigned to the *AlignerZ* audit contest(in which 150 Security researchers have participated) conducted a mitigation review on the smart contracts in the `contracts` repository from `Alignerz`, at commit hash `e4354a2407ba5358a1ab22298b5712aa224fbb20`.

Files
src/contracts/deploy/TokensDeployer.sol
src/contracts/nft/AlignerzNFT.sol
src/contracts/nft/ERC721A.sol
src/contracts/realYieldDistributor/RealYieldDistributor.sol
src/contracts/token/A26Z.sol
src/contracts/vesting/Alignerz.sol
src/contracts/vesting/feesManager/BiddingFeesManager.sol
src/contracts/vesting/feesManager/MergeSplitFeesManager.sol
src/contracts/vesting/TVSManager.sol
src/contracts/vesting/whitelistManager/WhitelistManager.sol
script/a26zBase.s.sol
script/a26zBaseSepolia.s.sol

Roles

- Owner: The owner of the protocol, is in charge of pausing and unpausing the contracts, launching projects, creating pools for the project, finalizing the bidding phase, setting project allocations, changing the vesting period divisor if needed and withdrawing funds that are stuck in the contract and profits

- Minter: Basically the `TVSManager` and `Alignerz` contracts that will be minting and burning the NFTs
- User: KOLs and bidders, some of the bidders will lose the bidding and get a refund, the winners will get their TVS which is mandatory to claim the project tokens

Executive Summary

The *AlignerZ* team partnered with *Dualguard* to conduct a security review of their smart contracts. *Dualguard* manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure. In order to achieve this, the *AlignerZ* made sure to fix the issues found during the review. The *Dualguard* team loved reviewing the *AlignerZ* codebase as the contracts follow simple logic, with correct and detailed ordering. The *AlignerZ* team was very responsive throughout the security review. They really care about the security of their protocol and the safety of their users. We believe that this collaboration was a great match.

Issues found

Severity	nb of Findings
High	1
Medium	2
Low	1
Info	10
Total	14

Findings

High

[H-1] NFT Split May Revert Due to Uninitialized Allocation Entries in `TVSCalculator`

Found by

0xSorryNotSorry and 0xAlix2

Description

The `calculateFeeAndNewClaimedSecondsForOneTVS` function does not populate amounts for already claimed flows, leaving entries as zero. During NFT splits, `computeSplitArrays` multiplies these zero entries by the split percentage and enforces `require(amount > 0)`. If the original NFT had any claimed flows, this requirement fails, causing the transaction to revert. NFTs with at least one claimed flow cannot be split, preventing users from reorganizing their holdings.

Root Cause

Uninitialized Allocation Entries in `TVSCalculator` for already claimed flows

Internal Pre-conditions

- User owns a TVS with 2 token flows or more with one of them being fully claimed

Attack Path

1- User tries to split his TVS but fails

PoC

```
    function test_Issue1_SplitFeeRevertsAfterAFlowIsClaimed() public {


        address alice = makeAddr("alice");
        vm.deal(alice, 1 ether);

        tvsManager.setAlignerz(address(this));


        vm.warp(1_000 days);
        uint256 nowTs = block.timestamp;

        uint256 nft1 = nft.mint(alice);
        uint256 nft2 = nft.mint(alice);

        tvsManager.setTVS(
            nft1,
            100 ether,
            1 days,
            nowTs - 2 days,
            IERC20(address(token)),
            0,
            false,
            0
        );

        tvsManager.setTVS(
            nft2,
            100 ether,
            365 days,
            nowTs - 1 days,
            IERC20(address(token)),
            0,
            false,
            0
        );

        token.transfer(address(tvsManager), 1_000 ether);

        // Merge into a multi flow TVS NFT.
        uint256[] memory ids = new uint256[](2);
        ids[0] = nft1;
        ids[1] = nft2;
```

```
44          vm.prank(alice);
45          uint256 mergedNftId = tvsManager.mergeTVS(ids);
46
47          vm.prank(alice);
48          tvsManager.claimTokens(mergedNftId);
49
50          TVSManager.Allocation memory alloc = tvsManager.getAllocationOf
                (mergedNftId);
51          assertTrue(alloc.claimedFlows[0], "expected flow0 claimed");
52          assertFalse(alloc.claimedFlows[1], "expected flow1 not claimed"
                );
53
54          uint256[] memory percentages = new uint256[](2);
55          percentages[0] = 5000;
56          percentages[1] = 5000;
57
58          vm.prank(alice);
59          vm.expectRevert(TVSCalculator.
                Splitting_Should_Not_Zero_Down_Amounts.selector);
60          tvsManager.splitTVS(percentages, mergedNftId);
61      }
```

Impact

User cannot split his TVS

Recommended Mitigation

Ensure `calculateFeeAndNewClaimedSecondsForOneTVS` sets amounts for claimed flows instead of leaving them uninitialized.

Status

Fixed.

Medium

[M-1] Global `vestingPeriodDivisor` Changes Can Break Valid Bid Updates

Found by

0xAlix2

Description

`updateBid` enforces that newVestingPeriod must be a multiple of the current global `vestingPeriodDivisor`. If the owner updates `vestingPeriodDivisor` after users have already placed bids, bidders may be unable to update only the amount of their bid without also changing the vesting period.

Recommended Mitigation

Apply the divisor check only when the vesting period is actually changed: - If newVestingPeriod == bid.vestingPeriod, skip the divisor validation. - Alternatively, snapshot vestingPeriodDivisor per bid or per project at creation time.

Status

Acknowledged

[M-2] `Alignerz - splitTVS can mint an NFT which is impossible to claim later`

Found by

0xSorryNotSorry

Description

This is more or less user error since the users can opt absurd percentages when splitting their TVSs. (like 1 and 9999)

The issue is: `splitTVS` can mint a child TVS where `claimedAmounts[i] > amounts[i]` for a flow that was partially claimed, and that permanently bricks `claimTokens` due to underflow.

It is because in `TVSCalculator.computeSplitArrays` the child flow is computed with floor math:

```
1          for (uint256 j; j < nbOfFlows; ) {
2   >>         uint256 amount = (baseAmounts[j] * percentage) /
       BASIS_POINT;
3              require(amount > 0, Splitting_Should_Not_Zero_Down_Amounts
                   ());
```

but the child already claimed one is computed with ceil math:

```
            else {
                claimedAmount = ceilDiv((baseClaimedAmounts[j] *
                    percentage), BASIS_POINT);
            }
```

So for small percentages (dust splits) and near fully claimed main flows, one can get `claimedAmount = amount + 1`. Then `TVSManager.getClaimableAmount` later computes as:

```
    function getClaimableAmount(Allocation memory allocation, uint256
        flowIndex) public view returns(uint256 claimableAmount) {
        uint256 vestingPeriod = allocation.vestingPeriods[flowIndex];
        uint256 vestingStartTime = allocation.vestingStartTimes[
            flowIndex];
        uint256 amount = allocation.amounts[flowIndex];
        uint256 claimedAmount = allocation.claimedAmounts[flowIndex];
        if (block.timestamp < vestingPeriod + vestingStartTime) {
>>>             claimableAmount = ((block.timestamp - vestingStartTime)
    * amount / vestingPeriod) - claimedAmount;
        } else {
>>>             claimableAmount = amount - claimedAmount;
        }
        return claimableAmount;
    }
```

and the lines with >>> might revert due to 1 wei difference.

PoC

Below is the test:

```
// SPDX-License-Identifier: MIT
pragma solidity =0.8.29;

import "forge-std/Test.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {ERC1967Proxy} from "@openzeppelin/contracts/proxy/ERC1967/
    ERC1967Proxy.sol";

import {AlignerzNFT} from "../src/contracts/nft/AlignerzNFT.sol";
import {TVSManager} from "../src/contracts/vesting/TVSManager.sol";
import {MockUSD} from "../src/MockUSD.sol";

contract TVSManager_SplitRounding_ClaimedAmountExceedsAmount_Test is
    Test {
    MockUSD internal usdc;
    AlignerzNFT internal nft;
    TVSManager internal tvsManager;

```

```
17      address internal alice;
18
19      function setUp() public {
20          alice = makeAddr("alice");
21
22          usdc = new MockUSD(); // 6 decimals
23          nft = new AlignerzNFT("AlignerzNFT", "AZNFT", "https://nft.
               Alignerz.bid/");
24
25          address impl = address(new TVSManager());
26          address proxy = address(
27              new ERC1967Proxy(
28                  impl,
29                  abi.encodeCall(TVSManager.initialize, (address(nft)))
30              )
31          );
32          tvsManager = TVSManager(payable(proxy));
33
34          nft.addMinter(address(tvsManager));
35
36          tvsManager.setAlignerz(address(this));
37
38          tvsManager.setTreasury(address(this));
39          tvsManager.setSplitFeeRate(0);
40      }
41
42      function
           test_PoC_SplitRoundingCanCreateClaimedAmountGreaterThanAmount_AndBrickClaims
           () public {
43
44
45          uint256 baseAmount = 3_000_007;
46          uint256 vestingPeriod = 365 days;
47
48          vm.warp(1_000 days);
49          uint256 startTime = block.timestamp - (vestingPeriod - 1);
50
51          uint256 nftId = nft.mint(alice);
52          tvsManager.setTVS(
53              nftId,
54              baseAmount,
55              vestingPeriod,
56              startTime,
57              IERC20(address(usdc)),
58              0,
59              false,
60              0
61          );
62
63          usdc.transfer(address(tvsManager), baseAmount);
64
```

```
65          vm.prank(alice);
66          tvsManager.claimTokens(nftId);
67
68          TVSManager.Allocation memory beforeSplit = tvsManager.
                getAllocationOf(nftId);
69          assertEq(beforeSplit.amounts.length, 1);
70          assertEq(beforeSplit.amounts[0], baseAmount);
71          assertEq(beforeSplit.claimedAmounts[0], baseAmount - 1);
72          assertFalse(beforeSplit.claimedFlows[0], "flow should be
                partial (not fully claimed)");
73
74          uint256[] memory percentages = new uint256[](2);
75          percentages[0] = 1;
76          percentages[1] = 9999;
77
78          vm.prank(alice);
79          (, uint256[] memory newNftIds) = tvsManager.splitTVS(
                percentages, nftId);
80
81          uint256 tinyNftId = newNftIds[0];
82          TVSManager.Allocation memory tiny = tvsManager.getAllocationOf(
                tinyNftId);
83          assertEq(tiny.amounts.length, 1);
84          assertEq(tiny.amounts[0], 300);
85          assertEq(tiny.claimedAmounts[0], 301);
86          assertFalse(tiny.claimedFlows[0], "still marked unclaimed, but
                claimedAmounts already exceeds amount");
87
88          vm.prank(alice);
89          vm.expectRevert(stdError.arithmeticError);
90          tvsManager.claimTokens(tinyNftId);
91      }
92  }
```

Recommended Mitigation

Make sure claimedAmount <= amount always

Status

Fixed

Low

[L-1] ClaimDeadline Boundary Condition Blocks Actions

Found by

0xAlix2

Description

The `claimRefund` and `withdrawPostDeadlineProfit` functions enforce strict inequalities around `claimDeadline`: - `claimRefund` requires `block.timestamp < claimDeadline` - `withdrawPostDeadlineProfit` requires `block.timestamp > claimDeadline` As a result, at exactly `block.timestamp == claimDeadline`, neither function can be executed, creating a small but real window where no action is allowed.

Recommended Mitigation

Consider using inclusive comparison for one of the functions to avoid the “gap”.

Status

Acknowledged

Informational

[I-1] Redundant Reset of ‘claimedAmount Causes Unnecessary State Writes

Found by

0xAlix2

Description

In `claimRealYields`, when the vesting period has fully elapsed, both `amount` and `claimedAmount` are reset to zero. Immediately after, `claimedAmount` is incremented again by `claimableAmount`. This reset of `claimedAmount` is redundant and serves no functional purpose.

Recommended Mitigation

Do not reset `claimedAmount`.

Status

Fixed

[I-2] Unused State Variable (`TVSManager`) Increases Contract Surface

Found by

0xAlix2

Description

The state variable `TVSManager` is declared and stored but never read or used anywhere in the contract. This introduces dead code and unnecessary storage usage.

Recommended Mitigation

Remove the `TVSManager` state variable entirely if it is not required.

Status

Fixed

[I-3] Non-Explicit Imports Reduce Readability

Found by

0xAlix2

Description

Several dependencies are imported using full file paths without named imports. This makes it unclear which symbols are actually used and increases audit and maintenance complexity.

Recommended Mitigation

Use explicit named imports for all external dependencies.

Status

Fixed

[I-4] Unused Struct Definition (Allocation) Increases Code Noise

Found by

0xAlix2

Description

The `Allocation` struct is defined in this contract, but is not used anywhere within it. This creates dead code and unnecessary complexity

Recommended Mitigation

Remove the `Allocation` struct entirely if it is not used in this contract.

Status

Fixed

[I-5] Missing Event Emission for `refundRoot` Update

Found by

0xAlix2

Description

`setProjectAllocations` emits events for each pool's merkleRoot update but does not emit an event when `refundRoot` is set. This creates inconsistent observability for critical allocation data.

Recommended Mitigation

Emit a dedicated event when `refundRoot` is set.

Status

Fixed

[I-6] Missing Limit on Number of NFTs in `splitTVS`

Found by

0xAlix2

Description

The `splitTVS` function allows a user to split a TVS NFT into an arbitrary number of new NFTs. There is currently no upper limit on how many NFTs a single TVS can be split into.

Recommended Mitigation

Introduce a maximum number of splits per NFT.

Status

Acknowledged

[I-7] Bad event emission on `mergeTVS`

Found by

0xSorryNotSorry

Description

The event is emitting `mergedTVSMemory.amounts` instead of `mergedTVS.amounts`. If the fee is > 0, these will differ.

Recommended Mitigation

Emit `mergedTVS.amounts` instead of `mergedTVSMemory.amounts` in the event.

Status

Fixed

[I-8] No Zero-value checks on `_vestingPeriod` in `launchDistribution`

Found by

0xSorryNotSorry

Description

The `_vestingPeriod` input is never checked to be GT 0 If it's set to zero, the claim functionality will be bricked due to division by zero.

Recommended Mitigation

require it to be > 0.

Status

Acknowledged

[I-9] No Zero-value nor Zero-address check in `setTVSAllocation`

Found by

0xSorryNotSorry

Description

`RewardProject storage rewardProject = rewardProjects[rewardProjectId];` -> This line should check the reward project is actually launched, not closed, and is not more than the rewardProjectCount

`address kol = kolTVS[i];` -> This line could check kol != 0

`rewardProject.kolTVSAddresses.push(kol);` -> This line could check for duplicate kols before pushing the addresses.

`uint256 amount = TVSamounts[i];` -> If not intentional, could check amount > 0

Recommended Mitigation

Add the recommended checks

Status

Acknowledged